Анализ данных с использованием языка Python

Пояснения к коду. Семинар 2

NumPy: массивы, индексация, векторизация

А. Н. Лата

Пояснения к ноутбуку `notebooks/seminar-02-numpy-arrays.ipynb`. Номер ячейки — её порядковый номер среди ячеек кода ноутбука, считая сверху. Для каждой ячейки приведён её код с номерами строк, а под ним — что делает каждая команда и что означает каждый аргумент именно в этой ячейке. Команды, разобранные в пояснениях к семинару 1, здесь только называются. Код совпадает с ноутбуком, только пробелы перед комментариями сокращены. Полные подписи функций со всеми параметрами — в справочнике `reference/numpy.md`.

1 Зачем NumPy юристу

Ячейка 1. Среднее по списку

```
1 amounts_list = [48_000, 250_000, 1_200_000, 75_000, 320_000, 15_000, ↩
  ↪ 640_000, 98_000, 410_000, 135_000]
2
3 sum(amounts_list) / len(amounts_list)  # средняя цена иска
```

- `amounts_list = [...]` — список Python из десяти цен исков.
- `sum(amounts_list)` — сумма элементов списка, `len(amounts_list)` — их число; частное — среднее.
- Последняя строка ячейки без `print`: Jupyter сам показывает её значение под ячейкой.

Ячейка 2. Скидка по списку

```
1 [amount * 0.7 for amount in amounts_list]  # цены после скидки 30 %
```

- `[amount * 0.7 for amount in amounts_list]` — генератор списка: для каждого элемента `amount` из списка вычислить `amount * 0.7` и собрать результаты в новый список. Это цикл `for` в одной строке.

Ячейка 3. Массив из списка

```
1 import numpy as np
2
3 amounts = np.array(amounts_list)
4 amounts
```

- `import numpy as np` — подключить NumPy, как на семинаре 1.
- `np.array(amounts_list)` — создать массив из списка; в выводе он показан как `array([...])`.

Ячейка 4. Среднее массива

```
amounts.mean()  # средняя цена иска
```

– `amounts.mean()` — метод массива: среднее всех элементов. Метод вызывается через точку после имени массива.

– `np.float64(319100.0)` — одно дробное число NumPy; пометка типа на значение не влияет, `print` её не показывает.

Ячейка 5. Скидка массивом

```
amounts * 0.7  # цены после скидки 30 %
```

– `amounts * 0.7` — каждый элемент умножается на 0.7; цикл не нужен. Результат — новый массив, `amounts` не меняется.

2 Создание и устройство массива

Ячейка 6. Тип массива

```
claims = np.array([48_000, 250_000, 1_200_000])
claims.dtype
```

– `claims.dtype` — тип элементов массива; `int64` — целые числа. Это атрибут, а не метод: без скобок.

Ячейка 7. Одно дробное число

```
np.array([48_000.50, 250_000, 1_200_000]).dtype
```

– `np.array([...]).dtype` — тип узнаётся сразу у нового массива, без переменной; `float64` — дробные числа.

Ячейка 8. Строки

```
np.array(['A40', 'A56', 'A41']).dtype
```

– `<U3` — строки Unicode длиной до трёх символов.

Ячейка 9. Логический тип

```
np.array([True, False, True]).dtype
```

– `True` и `False` — логические значения «да» и «нет», тип `bool`.

Ячейка 10. Смена типа

```
claims.astype(float)
```

– `claims.astype(float)` — новый массив того же содержания, но дробного типа; `float` — имя дробного типа Python. Исходный `claims` остаётся целым.

Ячейка 11. Дробные в целые

```
np.array([4.9, 5.5]).astype(int)
```

– `astype(int)` — перевод в целый тип отбрасывает дробную часть: 4.9 даёт 4, а не 5.

Ячейка 12. Картотека генератором

```
rng = np.random.default_rng(2026)

amounts = rng.integers(10_000, 1_000_000, size=10)  # цена иска, руб.
days = rng.integers(10, 200, size=10)  # срок рассмотрения, дней
outcomes = rng.integers(0, 2, size=10)  # 1 удовлетворён, 0 отказано

print(amounts)
print(days)
print(outcomes)
```

– `np.random.default_rng(2026)` — генератор с зерном 2026, как на семинаре 1.

– `rng.integers(10_000, 1_000_000, size=10)` — десять случайных целых от 10 000 до 999 999: правая граница не входит.

– `rng.integers(10, 200, size=10)` — сроки от 10 до 199 дней; `rng.integers(0, 2, size=10)` — исходы 0 или 1.

– Три массива созданы одним генератором подряд, поэтому каждый следующий продолжает его последовательность чисел. Если перезапустить только строку с `days`, числа будут другие.

– `print(amounts)` — массив печатается без `array(...)`: только числа через пробел.

Ячейка 13. Форма массива

```
amounts.shape, amounts.ndim, amounts.size
```

– `shape` — форма: размер по каждой оси; `ndim` — число осей; `size` — число элементов.

– Три значения через запятую в последней строке Jupyter показывает вместе, в круглых скобках.

Ячейка 14. Номера дел

```
np.arange(1, 11)  # номера дел
```

– `np.arange(1, 11)` — целые числа от 1 до 10: правая граница не входит.

Ячейка 15. Нули

```
np.zeros(10)
```

– `np.zeros(10)` — массив из десяти дробных нулей.

3 Арифметика и сравнения

Ячейка 16. Цены в тысячах

```
amounts / 1_000  # цены в тысячах рублей
```

– `amounts / 1_000` — каждый элемент делится на тысячу; результат дробный.

Ячейка 17. Неустойка

```
amounts * 0.001 * days  # неустойка за время рассмотрения, руб.
```

– `amounts * 0.001 * days` — сначала каждая цена умножается на 0.001, потом массив цен умножается на массив сроков попарно: первый элемент на первый, второй на второй.

Ячейка 18. Сравнение

```
amounts > 100_000
```

– `amounts > 100_000` — сравнение каждого элемента; результат — массив `True` и `False` той же длины.

Ячейка 19. Разные длины: намеренная ошибка

```
amounts + np.array([1, 2, 3])
```

– `np.array([1, 2, 3])` — массив из трёх элементов. Сложить его с массивом из десяти поэлементно нельзя: `ValueError` с формами обоих массивов, `(10,)` и `(3,)`.

4 Индексация и срезы

Ячейка 20. Первое и последнее дело

```
amounts[0], amounts[-1]  # первое и последнее дело
```

– `amounts[0]` — элемент с номером 0, первый; `amounts[-1]` — последний: минус считает с конца.

Ячейка 21. Срез

```
amounts[2:5]  # дела с третьего по пятое
```

– `amounts[2:5]` — элементы с номерами 2, 3 и 4: начало среза входит, конец нет.

Ячейка 22. Три последних

```
amounts[-3:]  # три последних дела
```

– `amounts[-3:]` — от третьего с конца до конца массива: пустое место после двоеточия значит «до конца».

Ячейка 23. Запись в срез

```
demo = np.array([10, 20, 30, 40])
part = demo[:2]
part[0] = 99
demo
```

– `part = demo[:2]` — срез из первых двух элементов: `[:2]` — от начала до номера 2 без него.

– `part[0] = 99` — запись в первый элемент среза. Срез — окно в данные `demo`, поэтому меняется и `demo`.

Ячейка 24. Запись в копию

```
demo = np.array([10, 20, 30, 40])
part = demo[:2].copy()
part[0] = 99
demo
```

– `demo[:2].copy()` — копия среза: отдельный массив. Запись в него `demo` не меняет.

5 Логические маски

Ячейка 25. Маска

```
mask = amounts > 100_000
mask
```

– `mask = amounts > 100_000` — результат сравнения сохраняется в переменную; это маска.

Ячейка 26. Отбор маской

```
amounts[mask]  # только крупные иски
```

– `amounts[mask]` — маска в квадратных скобках оставляет элементы, напротив которых `True`. Маска должна быть той же длины, что массив.

Ячейка 27. Сколько и какая доля

```
mask.sum(), mask.mean()  # число и доля крупных исков
```

– `mask.sum()` — сумма маски: `True` считается единицей, `False` — нулём, поэтому сумма — число отмеченных элементов.

– `mask.mean()` — среднее маски: доля отмеченных.

Ячейка 28. Маска по другому массиву

```
days[amounts > 100_000]  # сроки крупных исков
```

– `days[amounts > 100_000]` — маска построена по ценам, а отбирает сроки. Так можно, потому что позиции в `amounts` и `days` соответствуют одним и тем же делам.

Ячейка 29. Два условия

```
amounts[(amounts > 100_000) & (outcomes == 1)]  # удовлетворённые крупные ↩
↪ иски
```

– `outcomes == 1` — двойное равно: сравнение, а не присваивание.

– `(...) & (...)` — «и»: `True` там, где верны оба условия. Каждое условие — в своих скобках.

Ячейка 30. Отрицание

```
amounts[~mask]  # иски не дороже 100 000
```

– `~mask` — тильда переворачивает маску: `True` становится `False` и наоборот.

Ячейка 31. Или

```
amounts[(amounts < 50_000) | (amounts > 500_000)]  # мелкие и очень крупные иски
```

– Вертикальная черта между условиями — «или»: `True` там, где верно хотя бы одно из них.

Ячейка 32. Без скобок: намеренная ошибка

```
amounts[amounts > 100_000 & outcomes == 1]
```

– Без скобок Python выполняет `100_000 & outcomes` раньше сравнений, и выражение превращается в двойное сравнение массивов. Python пытается получить из массива одно «да» или «нет» — отсюда `ValueError` со словом ambiguous.

Ячейка 33. and вместо &: намеренная ошибка

```
amounts[(amounts > 100_000) and (outcomes == 1)]
```

– `and` — «и» для одиночных значений: он спрашивает у каждой стороны одно «да» или «нет». У маски их десять, поэтому та же ошибка. Для масок — только амперсанд.

Ячейка 34. Запись по маске

```
capped = amounts.copy()
capped[capped > 400_000] = 400_000
capped
```

– `capped = amounts.copy()` — копия картотеки: менять будем её.

– `capped[capped > 400_000] = 400_000` — маска слева от знака равенства: записать 400 000 только в отмеченные элементы, остальные не трогать.

Ячейка 35. Маска по строкам

```
courts = rng.choice(['A40', 'A56', 'A41'], size=10)  # суд по каждому делу
print(courts)
amounts[courts == 'A40']
```

– `rng.choice([...], size=10)` — десять случайных значений из списка кодов судов, с повторами; подробнее — на семинаре 3.

– `courts == 'A40'` — сравнение строк поэлементно; маска по судам отбирает цены.

6 Статистики

Ячейка 36. Сумма, минимум, максимум

```
amounts.sum(), amounts.min(), amounts.max()
```

– `sum()`, `min()`, `max()` — методы массива: сумма, наименьший и наибольший элемент.

Ячейка 37. Округление

```
round(amounts.mean())  # средняя цена, до рубля
```

– `round(amounts.mean())` — среднее, округлённое до целого; `round` без второго аргумента даёт целое число.

Ячейка 38. Доля единиц

```
outcomes.mean()  # доля удовлетворённых исков
```

– `outcomes.mean()` — среднее нулей и единиц — доля единиц, то есть доля удовлетворённых исков.

Ячейка 39. Среднее по маске

```
days[outcomes == 1].mean() - days[outcomes == 0].mean()  # разница средних ↩
↪ сроков, дней
```

– `days[outcomes == 1].mean()` — средний срок удовлетворённых исков: сначала отбор маской, потом среднее отобранного.

– Разница двух таких средних — на сколько дней в среднем дольше шли удовлетворённые иски.

7 Двумерный массив как таблица

Ячейка 40. Таблица из трёх массивов

```
table = np.array([amounts, days, outcomes]).T
table
```

– `np.array([amounts, days, outcomes])` — массив из списка трёх массивов: три строки по десять чисел.

– `.T` — транспонирование: строки становятся столбцами. Получается десять строк — дел — и три столбца.

Ячейка 41. Форма таблицы

```
table.shape, table.ndim
```

– `shape` — `(10, 3)`: десять строк, три столбца; `ndim` — две оси.

Ячейка 42. Строка

```
table[0]  # первое дело
```

– `table[0]` — один индекс выбирает строку: первое дело целиком.

Ячейка 43. Столбец

```
table[:, 1]  # столбец сроков
```

– `table[:, 1]` — до запятой строки, после — столбцы; двоеточие — все строки, 1 — второй столбец.

Ячейка 44. Среднее столбца

```
table[:, 1].mean()  # средний срок, дней
```

– `table[:, 1].mean()` — среднее выбранного столбца: средний срок.

Ячейка 45. Сумма по оси 0

```
load = np.array([[1200, 1350, 1420, 1510],
                 [ 800,  790,  910,  880],
                 [2100, 2250, 2300, 2390]])

load.sum(axis=0)  # итог по годам
```

– Строки 1–3 — двумерный массив из списка списков: три строки по четыре числа, суд на год. Пробелы внутри скобок выравнивают числа и на значения не влияют.

– `load.sum(axis=0)` — сумма вдоль оси 0, сверху вниз: по числу на столбец, итог по годам.

Ячейка 46. Среднее по оси 1

```
load.mean(axis=1)  # среднее по судам
```

– `load.mean(axis=1)` — среднее вдоль оси 1, слева направо: по числу на строку, среднее по судам.

Ячейка 47. Сумма без оси

```
load.sum()  # дел за все годы
```

– `load.sum()` — без `axis` сумма всех элементов таблицы.